

G-TORA: 차량 그래프에서 오프로딩을 예측하는 GNN

Graph Neural Network-Based Task Offloading and Resource Allocation for Scalable Vehicular Networks

Menghan Shao · Rongqing Zhang · Liuqing Yang (2025)

가변 차량 수와 V2V 링크를 그래프로 입력해, GA 라벨을 모사하는 지도학습 모델

문제: 차량 수와 연결이 계속 달라진다

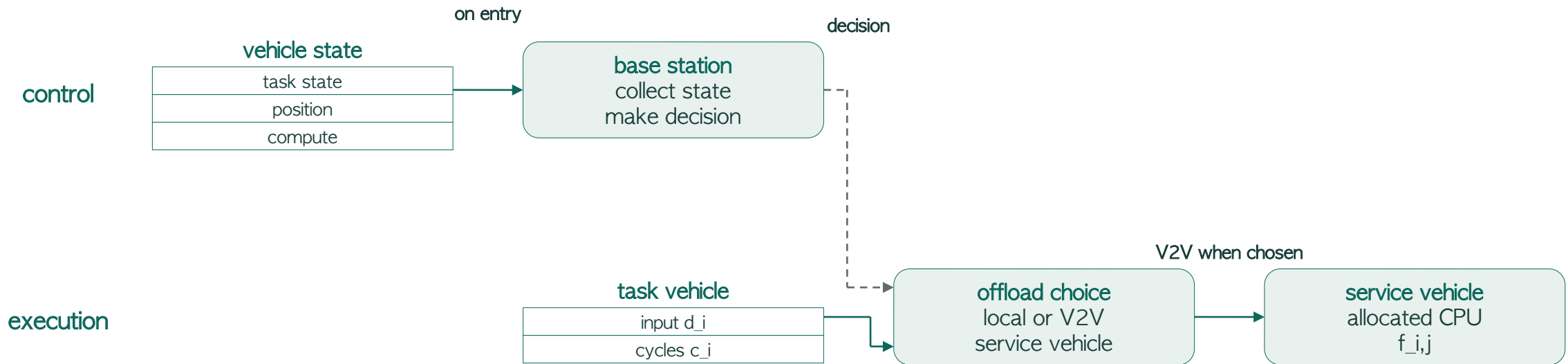
같은 기지국 범위 안에서도 task vehicle과 service vehicle의 수가 slot마다 바뀐다

- 차량은 들어오고 나가며, 가까운 차량과의 V2V 링크도 달라진다.
-
- task vehicle은 계산이 부족할 수 있고, service vehicle은 유휴 계산 자원을 제공한다.
-
- 고정 길이 벡터를 쓰는 MLP는 차량 수가 바뀌면 padding 또는 별도 처리가 필요하다.
-

입력의 핵심은 차량 목록만이 아니라 현재의 연결 구조다.

한 slot의 차량 오프로딩 구조

기지국은 정보를 모으고, task vehicle은 local 또는 가까운 service vehicle을 선택한다



기지국이 결정하지만, 계산 경로는 local 또는 task-to-service V2V다.

그래프의 정점과 엣지는 무엇을 뜻하는가

정점은 차량, 엣지는 차량 간 통신 링크이며 feature가 역할에 따라 다르다



그래프는 현재 연결을 담으므로, 노드 수가 달라도 같은 GNN 구조에 넣을 수 있다.

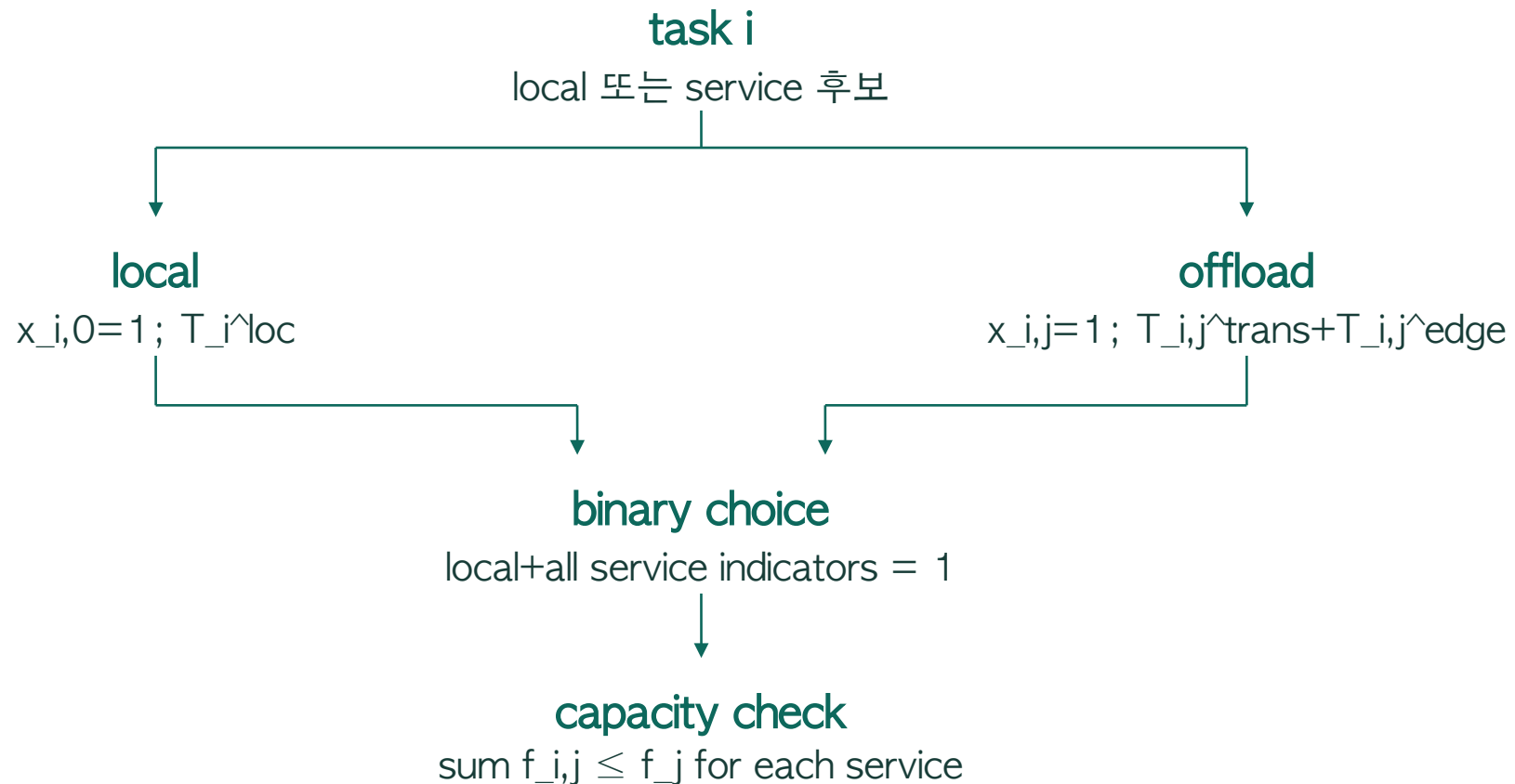
local과 offload의 지연 구성

경로	계산	통신	논문이 세는 지연
local	c_i / f_i	없음	T_i^{loc}
service vehicle	$c_i / f_{i,j}$	$d_i / R_{i,j}$	$T_{i,j}^{trans} + T_{i,j}^{edge}$
결과 반환	완료 뒤 전송	작다고 가정	지연에서 제외

오프로딩은 전송 시간이 추가되지만 더 큰 service CPU를 쓸 수 있다.

목표와 두 제약: 한 경로 선택, service 용량 준수

논문은 slot들의 총 response delay를 최소화한다



고정된 선택 뒤의 자원 배분

- 논문은 offloading action을 고정하면 남은 resource-allocation 문제가 convex가 된다고 말한다.
-
- 그래서 다음 절은 주로 binary offloading variable을 어떻게 예측할지에 집중한다.
-
- 하지만 fixed action 뒤의 allocation solver나 닫힌 형태의 해는 제시하지 않는다.

GNN이 자원 배분까지 어떤 계산으로 푸는지 논문만으로 재현할 수는 없다.

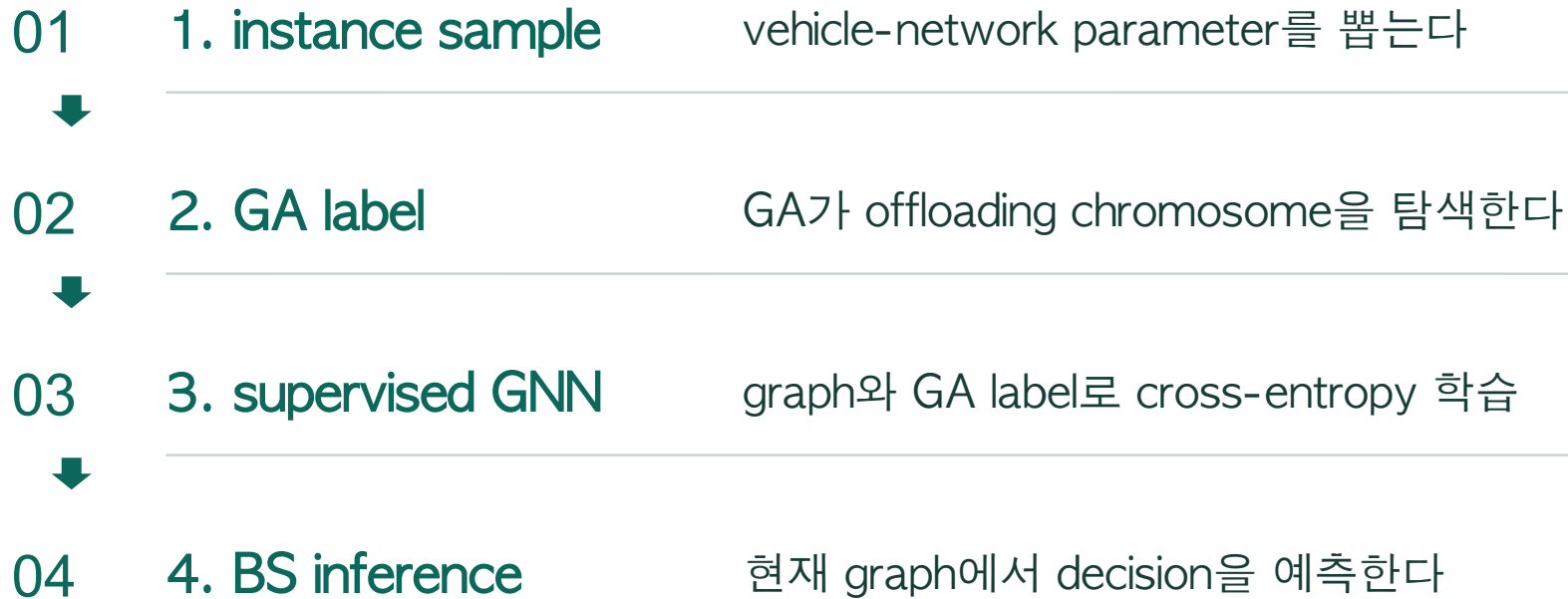
가변 차량 수에서 GNN을 쓰는 이유

방법	입력 크기 변화	논문이 말한 처리
고정 MLP	고정	적은 차량은 0 padding
고정 MLP	범위 초과	추가 task vehicle은 local 처리
GNN	그래프 정점·엣지 변화	현재 topology를 graph로 입력

논문의 scalability 주장은 가변 크기 graph 입력을 같은 GNN 구조로 처리한다는 뜻이다.

G-TORA의 라벨 생성·학습·추론 흐름

label 생성과 학습은 offline, 배포된 GNN의 결정 예측은 online이다



GA가 운용 중 매번 해를 찾는 구조가 아니라, GNN의 교사 역할을 한다.

GA는 오프로딩 라벨을 어떻게 만드는가

각 chromosome은 여러 task vehicle의 offloading decision 후보를 담는다

- | | | |
|----|--------------------|---|
| 01 | initial population | random decision chromosome 생성 |
| ↓ | | |
| 02 | evaluate | Eq. 6의 total response delay 계산 |
| ↓ | | |
| 03 | evolve | roulette selection, crossover, mutation |
| ↓ | | |
| 04 | best chromosome | GNN 학습용 decision label 반환 |

GA label은 exhaustive optimum이 아니라 휴리스틱 탐색 결과다.

GA fitness 식에는 재현상 빈틈이 있다

- 본문은 response delay가 낮을수록 fitness가 높다고 설명한다.

- Algorithm 1 line 5는 $\text{Eval}(Q^k) = \beta(1 - \beta)^{(k-1)}$ 를 인쇄한다.

- 그 식의 rank k 가 delay에서 어떻게 정해지는지는 논문이 설명하지 않는다.

GA label의 정확한 fitness 구현은 원문만으로 완전히 복원되지 않는다.

GNN 입력: node feature와 link feature

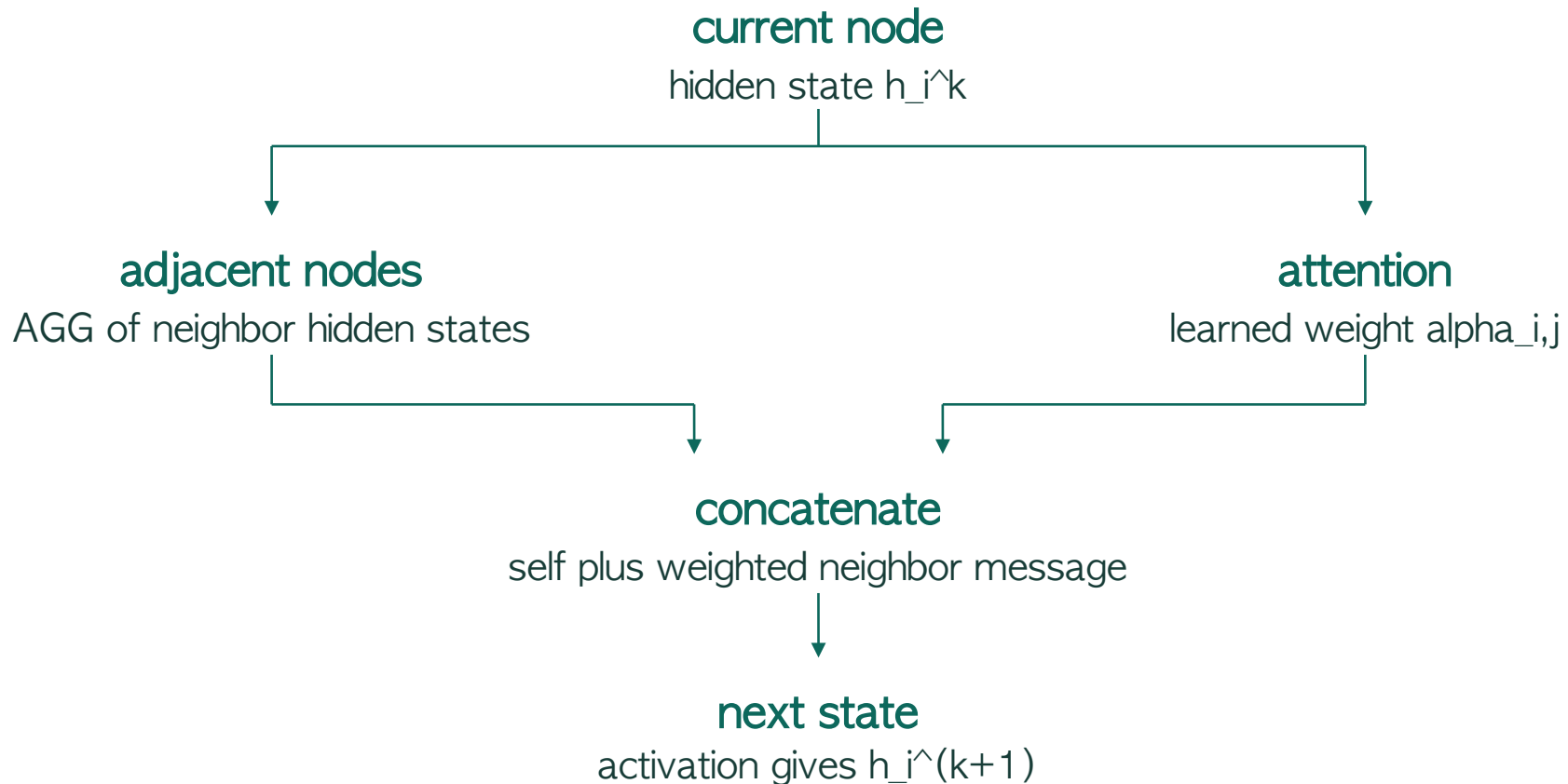
입력 전 task demand와 compute feature를 min-max scaling하고, edge weight는 SNR로 조정한다



feature는 역할별로 다르고, link 품질도 decision에 들어간다.

두 GNN layer의 이웃 정보 집계

각 정점은 자기 hidden state와 인접 정점의 message를 함께 업데이트한다



edge embedding에서 offloading decision까지

task node, service node, link feature를 합쳐 candidate edge의 decision을 분류한다



논문은 task-service 후보 link를 판단 단위로 결합해 MLP에 넣는다.

지도학습 training loop

GA label과 prediction의 cross-entropy가 GNN parameter를 갱신한다

01	batch graph	nodes, edges, attributes를 train loader에 저장
↓		
02	GNN forward	attention aggregation과 edge decision 출력
↓		
03	cross-entropy	prediction과 GA label을 비교
↓		
04	Adam update	backpropagation으로 weight 갱신
↓		
05	deploy	trained GNN을 base station에 저장

online 단계에서 탐색 보상을 누적하는 RL loop는 이 논문에 없다.

교육용 가상 trace: 한 task의 경로 비교

아래 수치는 논문 실험값이 아니라 Eq. 2-4를 읽기 위한 가정이다

가정	local	service vehicle	선택
$d=10\text{ MB}$, $c=10^9$ cycles	$f_i=1\text{ GHz}$	$\text{rate}=100\text{ MB/s}$ $f_{i,j}=4\text{ GHz}$	trained GNN이 service edge 선택
계산	$10^9 / 10^9$ $=1.00\text{ s}$	$10/100 + 10^9/(4 \times 10^9)$ $=0.10+0.25$	0.35 s
capacity	local 자원	$f_{i,j}=4\text{ GHz}$ 가 service f_j 이내	C1·C2 충족

좋은 V2V rate와 큰 배정 CPU가 전송 지연을 상쇄할 때 offload가 유리하다.

시뮬레이션 입력과 학습 설정

항목	논문 설정	항목	논문 설정
task data	{10,20,50,100} MB	task cycles	$[1,2] \times 10^9$
vehicle capacity	[3,5] GHz	bandwidth	10 MHz
transmit power	100 mW	distance	[20,500] m
GNN layers	2	embedding	256
epochs / optimizer	3000 / Adam	platform	Python 3.8, PyTorch

결과는 이 범위와 simulation platform에서 얻은 조건부 비교다.

비교 대상이 측정하는 trade-off

방법	논문에서의 역할	입력 크기 처리
GA	heuristic label generator	매 slot search
CH	local-search heuristic	local optimum 가능
MLP	fixed DNN baseline	0 padding 또는 excess local
OMAB	epsilon-greedy online bandit	탐색·활용 조절
GNN	GA-supervised proposal	current topology graph

낮은 processing delay와 빠른 inference는 서로 다른 측정값이다.

task vehicle 수를 5에서 10으로 늘린 비교

service vehicle 수는 4로 고정한 simulation이다

측정	관찰	읽을 때의 조건
loss	GNN은 약 0.2로 수렴	Fig. 4, 3000 epochs
processing	10대에서 GNN은 GA보다 약 17% 높음	CH보다 약 1% 높음
processing	GNN은 MLP보다 26% 낮음	OMAB보다 41% 낮음
inference	GNN은 0.03 s 미만	GA 14.664에서 31.016 s
total	GNN이 가장 낮게 보고됨	Fig. 7 조건에서만

GNN은 이 조건에서 total delay가 가장 낮게 보고됐다.

transmit power sweep의 결과

문제 크기가 고정된 상태에서 power를 0.5 W에서 1 W로 바꾼다

방법	0.5 W processing	1 W processing	inference
GA	1.056 s	0.690 s	크게 변하지 않음
CH	1.101 s	0.727 s	크게 변하지 않음
GNN	1.126 s	0.771 s	크게 변하지 않음
MLP	1.418 s	1.036 s	크게 변하지 않음
OMAB	1.840 s	1.163 s	크게 변하지 않음

power 증가로 통신 지연은 줄고, inference는 거의 유지됐다.

service compute capacity sweep

- 평균 computing capacity를 3 GHz에서 7 GHz로 올리면 모든 방법의 processing delay가 감소한다.
-
- GNN의 total delay는 0.923 s에서 0.817 s로 보고된다.
-
- GA와 CH의 total delay는 inference delay의 영향으로 비교적 안정적이라고 논문은 해석한다.

CPU가 빨라져도 전체 지연은 계산 지연과 decision inference 지연의 합으로 읽어야 한다.

이 논문에서 scalability가 성립하는 범위

- 동일한 GNN architecture가 다른 수의 graph node와 edge를 입력으로 받을 수 있다.
- 실험에서 직접 바꾼 범위는 task vehicle 5–10대이며 service vehicle은 4대로 고정했다.
- 논문은 arbitrary network size 성능, 통신 비용, 또는 formal scalability theorem을 제시하지 않는다.

scalability는 구조적 입력 적응성과 해당 simulation 범위의 결과로 제한해 읽어야 한다.

논문이 직접 밝힌 적용 한계

한계	왜 중요한가	논문이 제안한 이후 방향
GA supervision	많은 topology data와 label이 필요	semi- 또는 unsupervised GNN
slot 내 static channel	고속 이동의 link 변화를 충분히 반영 못함	real-time dynamics model
제외한 QoS	deadline·reliability를 보장하지 않음	QoS-aware offloading
제외한 task 구조	subtask dependency·multi-hop 미모델	future extension

G-TORA의 decision은 현재 model의 mobility·QoS 한계를 가진다.

G-TORA를 한 문장씩 다시 연결하기

01	network	task/service vehicle과 V2V link를 graph로 표현
↓		
02	objective	local 또는 offload response delay를 최소화
↓		
03	teacher	GA heuristic이 offline decision label 생성
↓		
04	learner	two-layer GNN이 link context로 label 모사
↓		
05	scope	simulation 범위와 static-slot 가정에서 해석

가변 차량 topology를 graph 입력과 GNN inference로 연결한다.